Swami Sahajanand College of Computer Science

B.C.A. SEM-VI [NEP]

Subject: ADVANCE JAVA
Major (Core) - 27197

UNIT 3

EXCEPTION AND JDBC CONNECTIVITY USING MS-ACCESS

- Exception Handling Fundamentals,
- Types of Exceptions.
- Try…catch Keyword, Multiple Catch Statements.
- Throw, Throws, Finally Keywords.
- JDBC Architecture
- Steps of Database Connectivity and Database Operation:
  Insert, Update, Delete
- Statement and Result Set Object
- Display Records using J Table Component

Exception Handling

During developing of any program we make some types of errors. Such as…

- Compile time errors
- Run time errors

What is Compile time error?

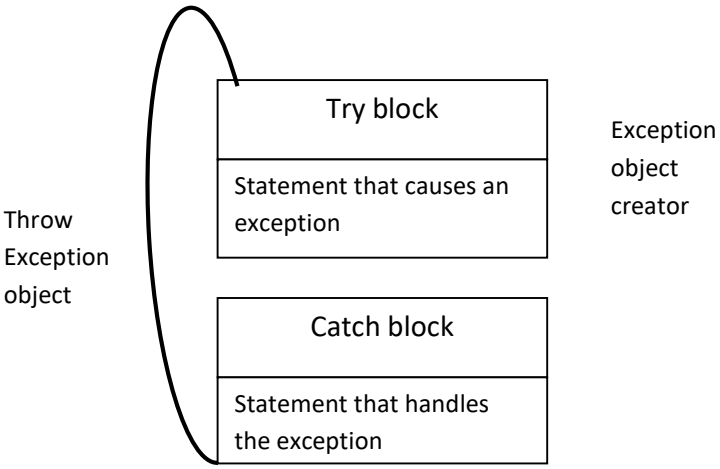
- Any syntax is incorrect in our program that time compiler generates errors are known as compile time errors.
- Such as missing semicolons.

What is Run time error?

- When we execute any program that time any errors occur is known as run time errors.
- It is also known as Exception.
- Some examples of Exception are….
    - Division by zero.
    - Stack overflow.
    - Null pointer assignment.
    - Arithmetic overflow.
    - Out-of-bounds array subscript.
- Exception is an event during program execution that stops the program from continuing normally.
- Java programming language supports exception handling with the 'try', 'catch', 'throw' and 'throws' keywords.
- Exceptions can be generated by the java run-time system or by the user code.
- Exception handling code basically consists of two parts.
- One to detect errors and to throw exceptions and the other to catch exceptions and to take appropriate actions.

Exception handling

- This mechanism provides four following steps to handle the exception.
- Find the problem. (Hit the exception).
- Inform that an error has occurred. (Thrown the exception)
- Receive the error information. (Catch the exception)
- Take corrective actions (Handle the exception)



Exception handling mechanism with diagram

What is try block?

- Code that is likely to generate an exception (error) is enclosed or written in a try block.
- In other words we can say statements that are likely to generate runtime error are enclosed in braces and preceded by the keyword try.
- Syntax:-

```
try
{
    code that might causes an error
}
```

What catch block?

- Catch statement is passed a single parameter, which is reference to the exception object thrown by the try block.
- If the catch parameter matches with the type of exception object, then the exception is caught and the statements in the catch block will be executed.
- Syntax:-

```
catch(Exception-type e)
{
     statement for managing exception
}
```

- Example of Exception Handling

```
Class demo
{
        Public static void main(String args[])
        {
```

```
                Int a=10,b=5,c=5,x,y;
                try
                {
                        x=a/(b-c);
                }
                Catch(ArithmeticException e)
                {
                        System.out.println("Division by Zero");
                }
                y=a/(b+c);
                System.out.println("Y = "+y);
        }
}
```

When we need multiple catch statement?

- Some times more than one exception could be generated by a single piece of code.
- To handle this type of situation, you can specify two or more catch statements, each catching a different type of exception.
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.
- After one catch statement executes, the others are bypassed, and execution are continues after the try/catch block.
- Example of Multiple catch statement

```
Class multicatch
 {
        Public static void main(String args[])
        {
                try
                {
                        Int a=args.lengh;
                        System.out.println("A:- "+a);
                        Int b=42/a;
                        Int c[]={1};
                        C[42]=99;
                }
                Catch(ArithmeticException e)
                {
                        System.out.println("Divide by Zero : "+ e);
                }
                Catch(ArrayInexOutOfBoundsException e)
                {
                        System.out.println("Array index is out of bound "+ e);
                }
                        System.out.println("After Exception complete");
```

```
        }
    }
```

Explain Throw, Throws & Finally keyword with example.

❖ **Throw**

- 'Throw' is a keyword that allows the user to throw an exception or any class that implements the 'throwable' interface.
- 'Throw' is used to indicate that exception has occurred.
- The operand of throw is an object of a class, which is derived from class Throwable.
- Syntax:
  throw Throwableinstance;
- Where, Throwableinstance must be an object of type Throwable or a subclass of Throwable.
- Simple types, such as int or char, as well as non-Throwable classes, such as String and Object, can not be used as exceptions.
- The flow of execution stops immediately after the throw statement.
- Example:

```
Public class demo
{
        Public static void main(String args[])
        {
                Throwable exc=new Throwable("Exception is generated");
                try
                {
                        System.out.println("We are now in try block");
                        throw exc;
                }
                catch(Throwable e)
                {
                        System.out.println("Exception is caught " + e);
                }
        }
}
```

Throws:

- 'Throws' keyword is used in method declarations that specify which exceptions are not handled within the method but rather passed to the next higher level of the program.
- If any method that might throw an exception, then possibility must be declared by throws statement.
- Throws clause lists the types of exceptions that a method might throw.
- Syntax:

```
method_type method_name() throws exception1, exception2,……exception
{
```

```
        // body of method
    }
```

Example:

```
Public class demo
{
    Public static void main(String args[]) throws Exception
    {
        int no=24,I,ans;
        for(i=0;i<=5;i++)
        {
            try
            {
                ans=no/(i+2);
                System.out.println("Answer := "+ans);
            }
            catch(ArithmeticException e)
            {
                System.out.println("Rethrowing the exception at i:= " + i);
                throw e;
            }
        }
    }
```

Finally:

- 'Finally' is a keyword that executes a block of statements regardless of whether a java exception, or run time error, occurred in a block defined previously by the 'try' keyword or not.
- 'Finally' block is the block in which return resources to the system and other statements for printing any message can be written.
  - Finally keyword includes …..
  - Closing a file
  - Closing a result set
  - Closing the connection established with the database.
- This block is optional but when included it is placed after the last catch block of a try.
- Finally block is always executed no matter whether exception is thrown or not.
- Either on of the catch {} or finally {} must be immediate after try {}.
- Control returns to the finally {}, when break or a return statement is executed in try {}.
- Example:

```
Public class demo
{
    Public static void main(String arg[])
    {
        Int a=25;
        try
```

```
                {
                        a=a/0;
                }
                catch(ArithmeticException e)
                {
                        System.out.println("Divide by Zero error");
                }
                finally
                {
                        System.out.println("Exception or No Exception it will be
                        executed");
                }
        }
}
```

Explain user defined Exception.

- Java provides built in exceptions to handle most common errors.
- Java also provides a way to create user defined exceptions, which are called custom exception or user exception.
- In user defined exception class user handles application specific problems.
- We can derive our new exception class from Exception or IOException base class.
- It is customary to give both a default constructor and a constructor that contains a detailed message.
- This can be done by creating a subclass of Exception.
  - Throwable()
- Construct a new throwable object with no detailed message.
  - Throwable(String message)
- Example:

```
Public class newexception extends Exception
{
        newexception(String s)
        {
                System.out.println("Unmatched exception is thrown out" + s);
        }
        Public static void main(String args[])
        {
                newexception e=new newexception("Hello");
                try
                {
                        throw e;
                }
                catch(Exception e)
                {
                        System.out.println(" Exception is caught := "+ e);
                }
        }
}
```

Checked Exceptions and Unchecked Exception

Checked Exception

- Checked exceptions are exceptions that are not runtime exceptions and are checked by the compiler.
- Thus the compiler does not require that you catch or specify runtime exceptions, although you can.
- Java has different types of exceptions, including I/O Exceptions, runtime exceptions, and exceptions of your own creation, to name a few.
- Runtime exceptions are those exceptions that occur within the Java runtime system.
- This includes arithmetic exceptions (such as when dividing by zero), pointer exceptions (such as trying to access an object through a null reference), and indexing exceptions (such as attempting to access an array element through an index that is too large or too small).
- Runtime exceptions can occur anywhere in a program and in a typical program can be very numerous.
- The cost of checking for runtime exceptions often exceeds the benefit of catching or specifying them.

Unchecked Exception

- Exceptions, which are not checked exceptions, are called unchecked exceptions.
- The RuntimeException class is one of the most important subclass of Exception class.
- It describes run-time problems that can commonly occur in programs.
- ArrayIndexOutOfBoundsException, ArithmaticException, NegativeArraySizeException are the examples of unchecked exceptions.

JDBC:

- JDBC stands for Java Database Connectivity.
- JDBC was developed by JavaSoft of Sun Microsystems.
- JDBC is a standard Java API for database-independent connectivity between the Java programming language and a wide range of databases
- It defines interfaces and classes for writing database applications in Java by making database connections. Using JDBC you can send SQL, PL/SQL statements to almost any relational database.
- JDBC is a Java API for executing SQL statements and supports basic SQL functionality. It provides RDBMS access by allowing you to embed SQL inside Java code. Since nearly all relational database management systems (RDBMSs) support SQL, and because Java itself runs on most platforms, JDBC makes it possible to write a single database application that can run on different platforms and interact with designed different DBMSs.
- Java Database Connectivity is similar to Open Database Connectivity (ODBC) which is used for accessing and managing database, but the difference is that JDBC is specifically for Java programs, whereas ODBC is not depended upon any language.

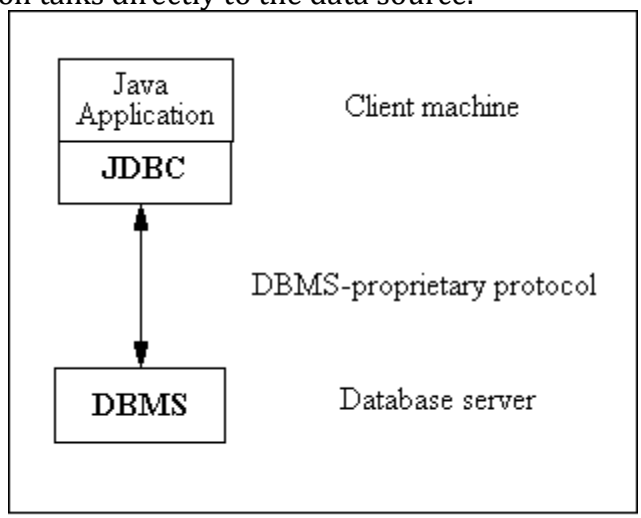
- In short JDBC helps the programmers to write java applications that manage these three programming activities:
- Connect to a data source, like a database.
- Send queries and update statements to the database.
- Retrieve and process the results received from the database in answer to your query.

➢ JDBC Architecture

- The JDBC API supports both two-tier and three-tier processing models for database access.

In the two-tier **Two-tier Architecture for Data Access or Two-tier database design**

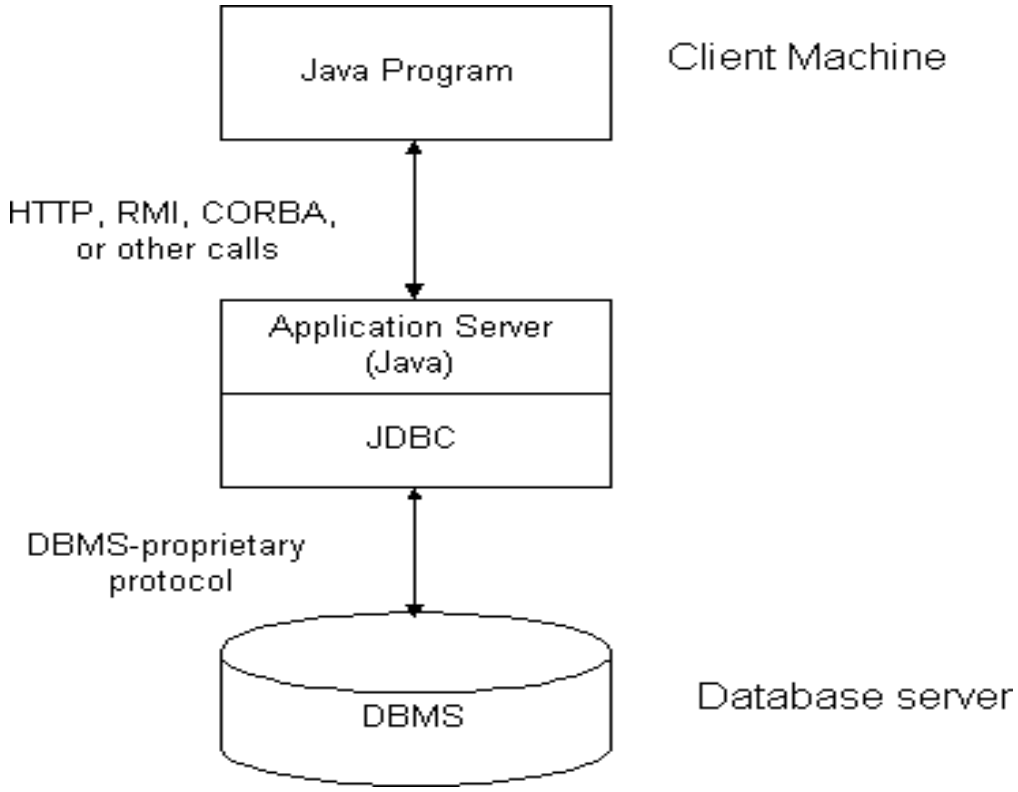
- model, a Java application talks directly to the data source.



Two-tier model

- This requires a JDBC driver that can communicate with the particular data source being accessed.
- A user's commands are delivered to the database or other data source, and the results
- of those statements are sent back to the user.
- The data source may be located on another machine to which the user is connected via a network. This is referred to as a client/server configuration, with the user's machine as the client, and the machine housing the data source as the server.
- The network can be an intranet, which, for example, connects employees within a corporation, or it can be the Internet.

Three-tier Architecture for Data Access



- In the three-tier model, commands are sent to a "middle tier" of services, which then sends the commands to the data source.
- The data source processes the commands and sends the results back to the middle
- tier, which then sends them to the user.
- MIS directors find the three-tier model very attractive because the middle tier makes it possible to maintain control over access and the kinds of updates that can be made to corporate data.
- Another many cases, the three-tier architecture can provide performance advantages
- advantage is that it simplifies the deployment of applications. Finally, in

Three-tier Architecture for Data Access.

- Until recently, the middle tier has often been written in languages such as C or C++, which offer fast performance. However, with the introduction of optimizing compilers that translate Java byte code into efficient machine-specific code and technologies such as Enterprise JavaBeans™, the Java platform is fast becoming the standard platform for middle-tier development. This is a big plus, making it possible to take advantage of Java's robustness, multithreading, and security features.
- With enterprises increasingly using the Java programming language for writing

- server code, the JDBC API is being used more and more in the middle tier of a three-tier architecture. Some of the features that make JDBC a server technology are its support for connection pooling, distributed transactions, and disconnected rowsets. The JDBC API is also what allows access to a data source from a Java middle tier.

Advantage

Provide Existing Enterprise Data

- Businesses can continue to use their installed databases and access information even if it is stored on different database management systems

Simplified Enterprise Development

- The combination of the Java API and the JDBC API makes application development easy and cost effective

Zero Configuration for Network Computers

- No configuration is required on the client side centralizes software maintenance. Driver is written in the Java ,so all the information needed to make a connection is completely defined by the JDBC URL or by a DataSource object. DataSource object is registered with a Java Naming and Directory Interface (JNDI) naming service.

Full Access to Metadata

- The underlying facilities and capabilities of a specific database connection need to be understood. The JDBC API provides metadata access that enables the development of sophisticated applications.

Disadvantage

- The process of creating a connection, always an expensive, time-consuming operation, is multiplied in these environments where a large number of users are accessing the database in short, unconnected operations.
- Creating connections over and over in these environments is simply too expensive.
- When multiple connections are created and closed it affects the performance.

Steps to create JDBC application

There are 5 steps to connect any java application with the database in java using JDBC. They are as follows:

- Register the driver class
- Creating connection
- Creating statement
- Executing queries
- Closing connection

1) Register the driver class

- The forName() method of Class class is used to register the driver class.
- This method is used to dynamically load the driver class.

Syntax :

public static void **forName**(String className)throws ClassNotFoundException

Example to register the OracleDriver class Class.forName
("oracle.jdbc.driver.OracleDriver");

2) Create the connection object

- The getConnection() method of DriverManager class is used to establish connection with the database.

Syntax:

- public static Connection **getConnection**(String url)throws SQLException
- **public static** Connection **getConnection**(String **url**,String **name**,String **passwor d**) **throws** SQLException

Example to establish connection with the Oracle database

Connection con=DriverManager.getConnection("jdbc:oracle:thin:@localhost:15 21:xe","system","password");

3) Create the Statement object:

- The createStatement() method of Connection interface is used to create statement.
- The object of statement is responsible to execute queries with the database.

Syntax

public **Statement createStatement()**throws **SQLException**

Example

Statement stmt=con.**createStatement**();

4) Execute the query

- The executeQuery () method of Statement interface is used to execute queries to the database.
- This method returns the object of ResultSet that can be used to get all the records of a table

Syntax

public ResultSet executeQuery(String sql)**throws** SQLException

Example

```
ResultSet rs=stmt.executeQuery("select * from emp");
while(rs.next())
{
    System.out.println(rs.getInt(1)+"  "+rs.getString(2));
}
```

5) Close the connection object

- By closing connection object statement and ResultSet will be closed automatically.
- The close() method of Connection interface is used to close the connection.

Syntax

public void **close**()throws SQLException

Example

```
con.close();
```

Statement and ResultSet Object

- In Java, database connectivity is performed using JDBC. JDBC provides several classes and interfaces to interact with databases.
- Two important interfaces used in JDBC are Statement and ResultSet.
- **Statement** is used to execute SQL queries.
- **ResultSet** is used to store and process the data returned from the database.
- These objects help Java programs communicate with databases such as MySQL, Oracle Database, etc.

Statement Object

Definition

- A **Statement** object is used to send **SQL queries** to the database and execute them.
- It belongs to the java.sql package.

Syntax

```
Statement st = con.createStatement();
```

Where:

con = Connection object
createStatement() = method used to create a Statement object

Methods of Statement Object

Method	Description
`executeQuery(String sql)`	Executes SELECT query and returns ResultSet
`executeUpdate(String sql)`	Executes INSERT, UPDATE, DELETE queries
`execute(String sql)`	Executes any SQL command
`close()`	Closes the statement object

ResultSet Object

Definition

- A **ResultSet** object is used to store the **data returned from a SELECT query**.
- It acts like a **table of data** retrieved from the database.
- The ResultSet object also belongs to the java.sql package.

Syntax

```
ResultSet rs = st.executeQuery("SELECT * FROM student");
```

Where:

rs = ResultSet object
executeQuery() returns the result of the SQL query.

Methods of ResultSet Object

Method	Description
`next()`	Moves cursor to next record
`getInt(column)`	Gets integer value
`getString(column)`	Gets string value
`getFloat(column)`	Gets float value
`close()`	Closes ResultSet

Advantages
- Easy execution of SQL queries
- Allows retrieval of database records
- Helps process large amounts of data
- Provides methods for navigating records

Display Records using J Table Component

- JTable is a component in Java Swing used to display and edit two-dimensional tabular data (rows and columns). It is commonly used in desktop applications to show structured data such as student records, employee lists, product tables, etc.
- JTable is part of the javax.swing package and provides many features like sorting, selection, editing cells, and scrolling.

Class Declaration
- The JTable class is declared as:
- public class JTable extends JComponent
- It inherits properties from the JComponent class.

Constructors
- Some commonly used constructors are:
  JTable()
  Creates an empty table.

  JTable table = new JTable();
  JTable(int rows, int columns)

  Creates a table with a specified number of rows and columns.
  JTable table = new JTable(5, 3);

JTable(Object[][] data, Object[] columnNames)
Creates a table with initial data and column names.

Features

- Some important features include:
- Displays data in rows and columns
- Allows cell editing
- Supports row and column selection
- Can be connected with database data
- Supports scrolling using JScrollPane
- Allows sorting and filtering

Methods

- Some useful methods of JTable are:

Method	Description
getRowCount()	Returns number of rows
getColumnCount()	Returns number of columns
getValueAt(row, column)	Gets value of a specific cell
setValueAt(value, row, column)	Sets value in a cell
setRowHeight(int height)	Sets height of rows
setEnabled(boolean)	Enables or disables editing

Example:

```
table.setRowHeight(30);
```

Advantages

- Easy way to display structured data
- Supports editing and selection
- Can integrate with databases
- Provides built-in GUI table structure

Applications

- JTable is used in many applications such as:
- Student Management System
- Employee Database System
- Billing Software
- Inventory Management System